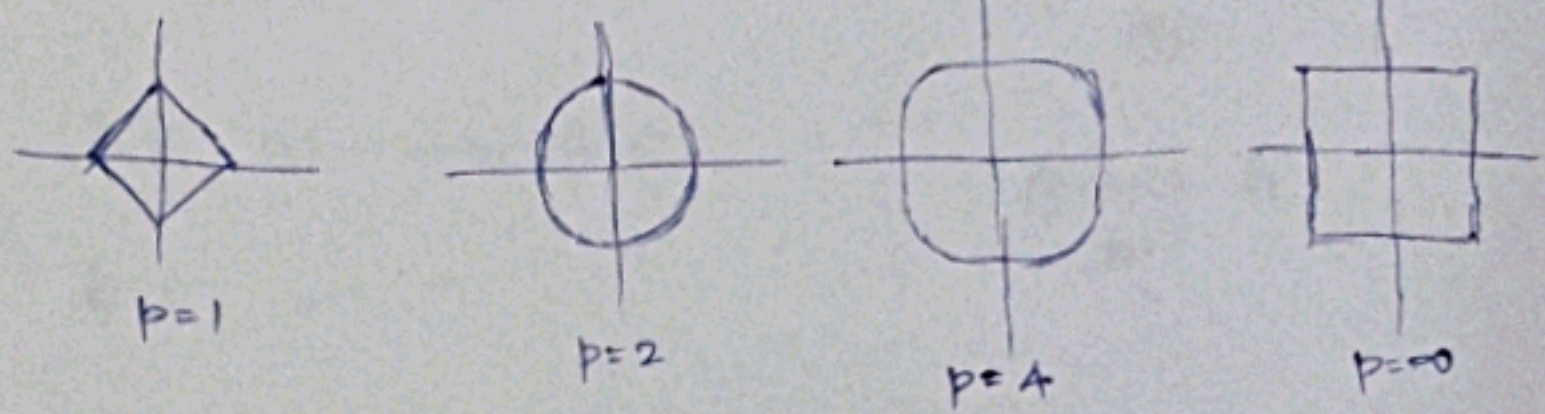
Deep Learning - Mid term exam

(Summary)

p-norm ($p \geq 1$) :- $\|x\|_p := \left(\sum_{i=1}^{n} |x_i|^p \right)^{1/p}$



Baye's theorem :-

$$P(x|y) = \frac{P(y|x) \cdot P(x)}{P(y)}$$

$P(x|y)$ ↙ Posterior

$P(y|x)$ ↗ Likelihood

$P(x)$ → Prior

$P(y)$ → observation

$$\text{Posterior} \propto \text{Likelihood} \times \text{Prior}$$

Gaussian distribution :-

(Univariate) $X \sim \mathcal{N}(\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \cdot e^{\frac{-(x-\mu)^2}{2\sigma^2}}$

(Multivariate) $\mathbf{x} \sim \mathcal{N}(\mu, \Sigma) = \frac{1}{\sqrt{(2\pi)^n \det \Sigma}} \cdot e^{-(x-\mu)^T \Sigma^{-1} (x-\mu)}$

↳ covariance matrix

Precision parameter : $\beta = \Sigma^{-1}$

→ covariance measures spread (more covariance ; more uncertainty)

→ Precision measures concentration (more precision ; tighter distribution ; less uncertainty)

KL Divergence :-

$$\text{Entropy :- } H(P) = -\sum_{x=1}^{n} P_x \times \log(P_x)$$

$$\text{Cross Entropy :- } H(P;Q) = -\sum_{x=1}^{n} P_x \cdot \log(q_x)$$

$$\text{KL Divergence :- } D(P\|Q) = H(P;Q) - H(P)$$

$$= -\sum_{x=1}^{n} P_x \log\left(\frac{P_x}{q_x}\right)$$

Classical V/s Bayesian estimation :-

Classical	Bayesian
Estimation parameter is an unknown deterministic quantity.	Estimation parameter is a random variable.
No prior (all values are equally likely)	Prior is used
Likelihood is optimized	Posterior is optimized
Poor performance whenever the parameter is not uniformly distributed.	Poor performance whenever the prior is incorrect.

(MLE) - No bias

(MAP) - Bias is built-in within prior.

Taylor Expansion :-

$$f(x) = f(a) + \frac{(x-a)}{1!} f'(a) + \frac{(x-a)^2}{2!} f''(a)$$

Classification :-

Binary Classification :- $K = 2$; $C_1 = +1$; $C_2 = -1$

Posterior probability for binary classification is,

$$Pr(y_i = +1 \mid x_i) = \frac{Pr(x_i \mid y_i = +1) \times Pr(y_i = +1)}{Pr(x_i \mid y_i = +1) \cdot Pr(y_i = +1) + Pr(x_i \mid y_i = -1) \cdot Pr(y_i = -1)}$$

$$\left\{ \begin{array}{l} a \rightarrow \text{log posterior ratios / (or) posterior odds} \\ a = \ln \left(\dfrac{Pr(x_i \mid y_i = +1) \cdot Pr(y_i = +1)}{Pr(x_i \mid y_i = -1) \cdot Pr(y_i = -1)} \right) \end{array} \right\}$$

$$Pr(y_i = +1 \mid x_i) = \frac{1}{1 + e^{-a}} = \sigma(a)$$

↳ sigmoid function.

Similarly, Posterior probability for K-class classification is,

$$Pr(y_i = C_k \mid x_i) = \frac{Pr(x_i \mid y_i = C_k) \cdot Pr(y_i = C_k)}{\sum_j Pr(x_i \mid y_i = C_j) \cdot Pr(y_j = C_j)}$$

$$= \frac{e^{a_k}}{\sum_i e^{a_j}}$$

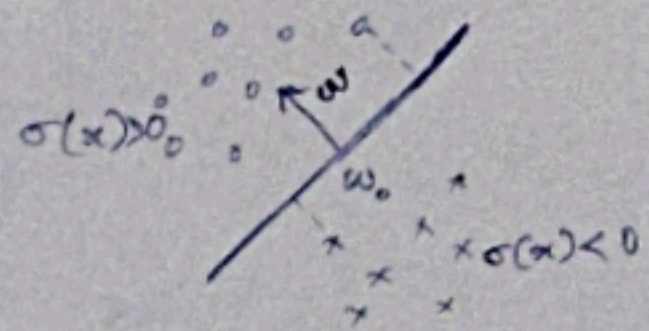
↳ Softmax function

Classifying linearly separable data

Hyperplane projection :- $(x - w_0)^T w = x^T w - w_0^T w$

$= x^T w - w_0$

[w_0 - shift of origin
w - normal to the hyperplane]



VC dimension :-

VC dimension :- $|\text{Test Error} - \text{Training Error}| \leq$

$$O\left(d_{vc} \frac{\log \frac{n}{d_{vc}}}{n}\right)^{1/2}$$

→ Talks about optimal capacity
→ below which, it is underfitting
→ beyond which, it is overfitting.

Single layer perceptron :-

All linearly separable data can be modelled using single layer perceptron. (No need of activation functions, as there is no non-linearity needed).

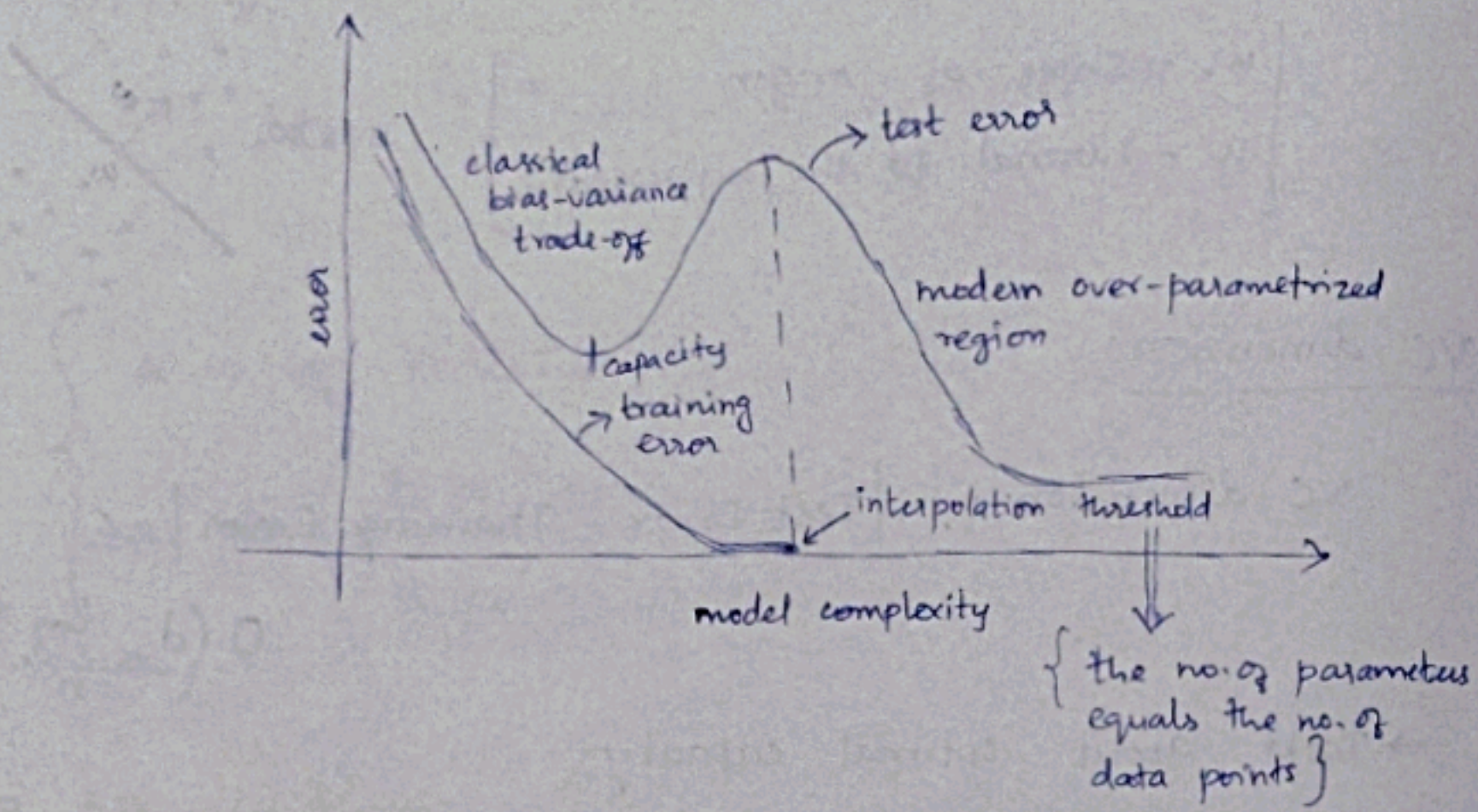
Multi Layer perceptron :-

XOR cannot be modelled using single layer perceptron as the data is not linearly separable.

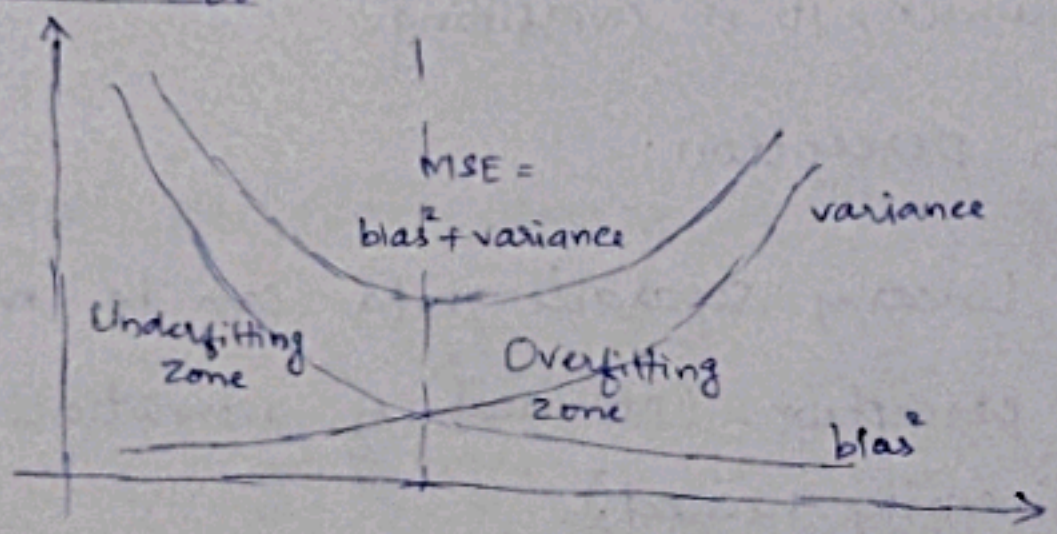
Double Descent :-

→ overfitting can be overcome with deeper networks

→ As the no. of parameters (depth) increases, the overfitting is smoothened.



Bias - Variance trade-off :-



$$\text{Bias} = E[y - y_t]$$

$$\text{Variance} = V[y]$$

$$MSE = E[(y - y_t)^2]$$

$$= \text{Bias}^2 + \text{Var}(y) + \text{Var}(y_t)$$

No Free Lunch theorem :-

No model can be a universal model that fits any unseen data well. As the unseen dataset can have different statistics, there can exist a dataset where any model can perform poorly.

So, there is a tradeoff b/w accuracy and robustness.

Inductive Bias :-

If some statistics of the entire data is known, use that information in addition to training data to optimize the model.

Grokking :-

Generalization of ova parametrized neural networks by over ~~pass~~ training.

Weight decay - Addition of a small term in the loss function, pushes the weights closer to zero.

Universal Function Approximation Theorems :-

* Stone - Weierstrass :-

  Every continuous function can be approximated by polynomials

* Andrew - Barron :-

  Lipshitz function can be approximated by infinite width 1-layer NN.

* Cybenko :-

  Lipshitz function can be approximated by infinite width 1-layer sigmoid NN.

Hornik - Stinchcombe - White :-

Lipshitz functions can be approximated by finite width multi-layer NN.

Lu - Pu - Klang - Hu - Wang :-

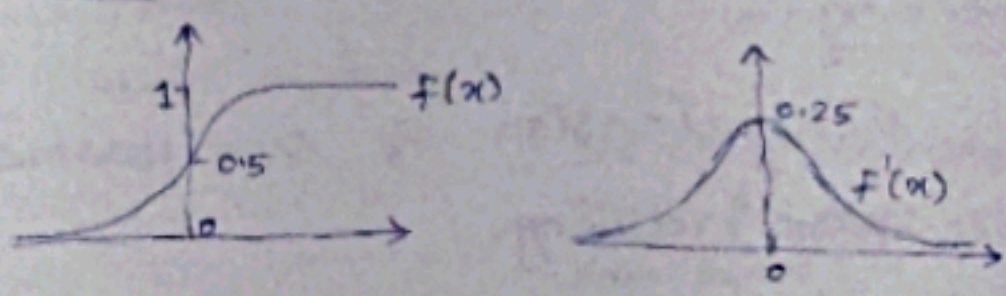
Lipshitz functions can be approximated by finite width multi-layer ReLU NN.

Kolmogorov - Arnold :-

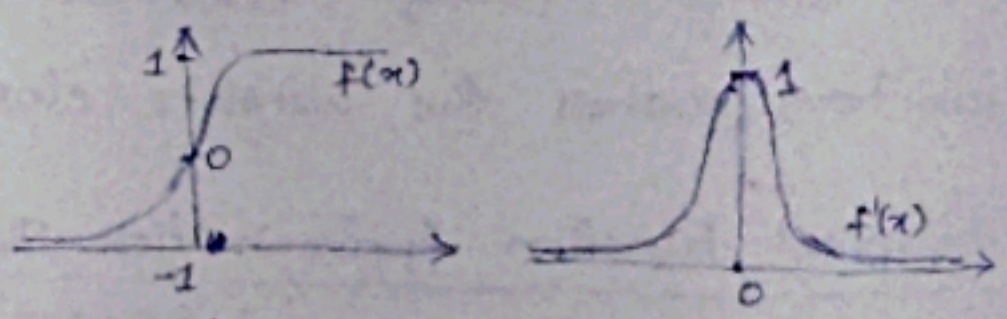
Lipshitz functions can be approximated by 2-layer NN

Some common activation functions :-

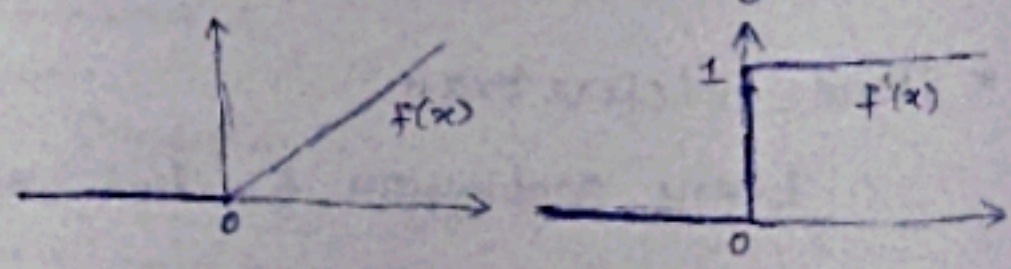
Sigmoid $\frac{1}{1+e^{(-x)}}$



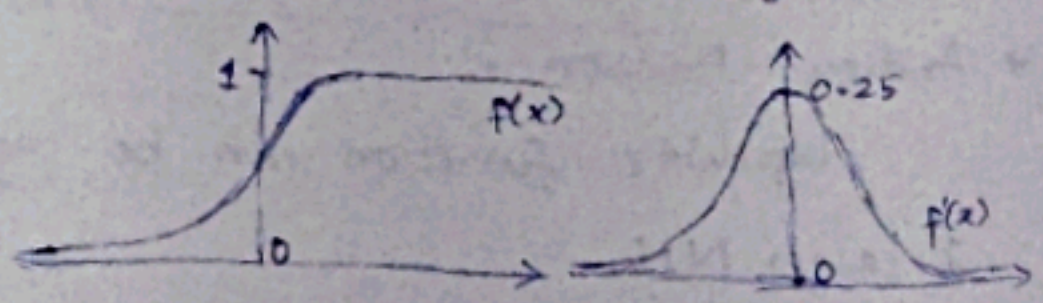
Tan h $\frac{1-e^{(-2x)}}{1+e^{(-2x)}}$



ReLU $\max(x, 0)$



Softmax $\frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$



Leaky ReLU

ELU

SeLU

Gradient Descent :-

Taylor approximation in higher dimension

Gradient :- Linear approximation of a function at a point

Hessian :- Curvature of a function at a point.

$$L(w+\varepsilon) \approx L(w) + \nabla L(w)^T (w+\varepsilon - w)$$

For, $L(w+\varepsilon) - L(w) < 0$,

pick ε opposite to $\nabla L(w)$

Back propagation :-

$$\text{Loss} := L\{ \hat{y} ; y \}$$

$$\text{Loss} = L\{ \overbrace{f_2(\overbrace{W_2 \underbrace{f_1(W_1 x)}_{a_1}}^{b_2})}^{\hat{y}} ; y \}$$

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial b_2} \times \frac{\partial b_2}{\partial W_2} = \frac{\partial L}{\partial b_2} \times \frac{\partial b_2}{\partial a_2} \times \frac{\partial a_2}{\partial W_2}$$

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial b_2} \times \frac{\partial b_2}{\partial a_2} \times \frac{\partial a_2}{\partial W_1} = \frac{\partial L}{\partial b_2} \times \frac{\partial b_2}{\partial a_2} \times \frac{\partial a_2}{\partial b_1} \times \frac{\partial b_1}{\partial a_1} \times \frac{\partial a_1}{\partial W_1}$$

compute, $g = \nabla_{\text{output}} L$

At layer i, $g = f_i' W_{i+1}^T g$

Initialization:-

* All weights should not be initialized as a same value, because, all the outputs of the neurons will be the same, and gradients will be the same, no learning occurs, and the loss will never converge.

* Xavier initialization (Sigmoid, Tanh)

$$u\left(\pm\sqrt{6/(n_{l+1}+n_{l-1})}\right)$$

* Kaiming He (ReLU)

$$N\left(0, 2/n_{l-1}\right)$$

* LeCun (SeLU)

$$u\left(\pm\sqrt{3/n_{l-1}}\right)$$

* Random Orthogonal matrix (RNN)

Autograd / Auto diff:-

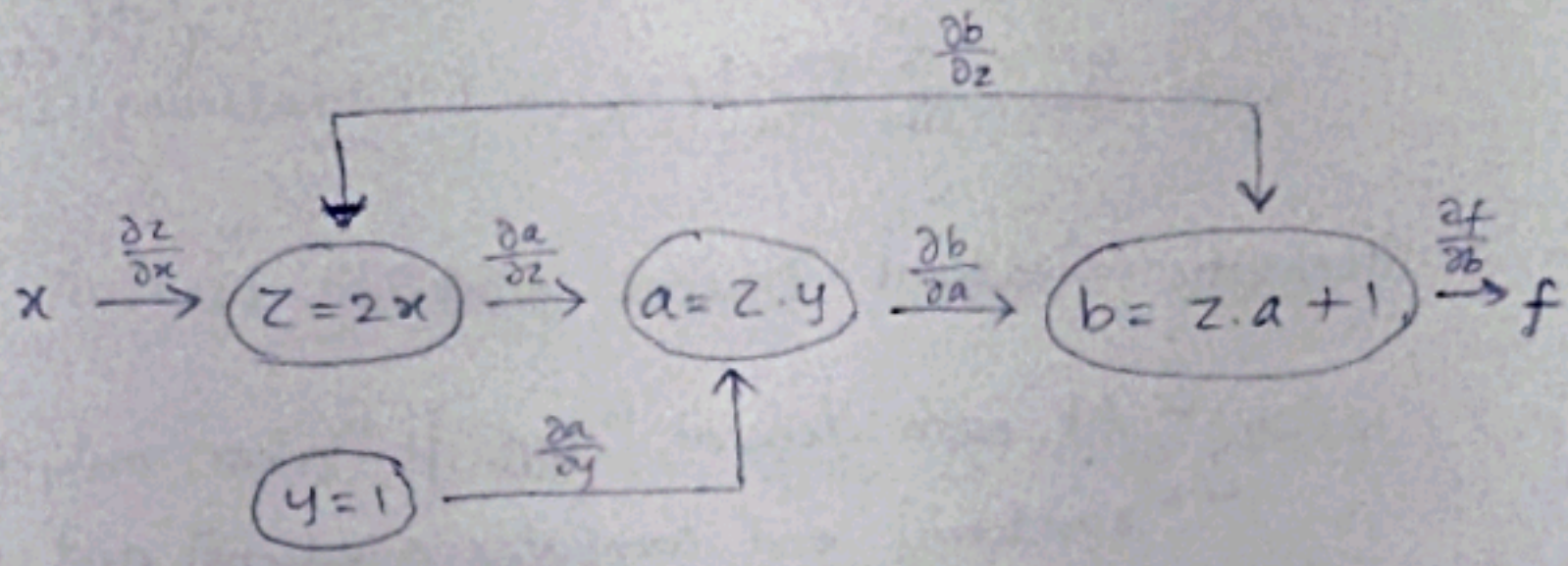
* Gradient of a function is computed using autograd which is exact gradient not numerical gradient (in programming).
* Works only for acyclic neural networks
  - ↳ $f(x) = x^2$
    $f'(x) = 2x$ (exact)
    $f'(x) = x_1^2 - x_2^2$ (numerical) $x_1 - x_2$
* Forward pass:- compute output value of all nodes
* Backward pass:- compute the gradients using the forward pass values and chain rule of derivatives.
* Implemented using computational graphs.

computational graphs:-

A directed acyclic graph representing all the computations between input and output.

→ Dynamic (Loops and Conditional statements)

→ Static



$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial b} \times \frac{\partial b}{\partial x}$$

$$= \frac{\partial f}{\partial b}\left[\frac{\partial b}{\partial z} \cdot \frac{\partial z}{\partial x} + \frac{\partial b}{\partial a} \cdot \frac{\partial a}{\partial x}\right]$$

$$= \frac{\partial f}{\partial b}\left[\frac{\partial b}{\partial z} \cdot \frac{\partial z}{\partial x} + \frac{\partial b}{\partial a}\left(\frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial x} + \frac{\partial a}{\partial y} \cdot \frac{\partial y}{\partial x}\right)\right]$$

* Autograd computes ~~the~~ gradient through back propagation
* Gradient flows from one node to another
* Vanishing and Exploding gradients are a problem in this.
* Auto grad fails for non-differentiable and improper computational graphs.

Gradient Descent :-

* Batch Gradient Descent (GD)

- Takes the whole dataset to compute gradient of the loss function. - convergence rate $\{O(1/n)\}$

$$g_n = \sum_{i=1}^{N} \nabla_{w_n} L_i / N$$

* Stochastic Gradient Descent (SGD)

- At each descent step, it takes only one datapoint randomly and computes gradient and converges.

$$w_{n+1} = w_n - \eta \nabla_w L_j \quad ; j = \{1, 2, 3, \cdots, N\}$$

- If works well for correllated data samples
  ↳ converges faster.

- Uncorrellated samples (noisy gradient) leads to divergence.

- convergence rate - $\{O(1/\sqrt{n})\}$

* Stochastic Gradient Descent mini-batch

- Instead of one datapoint, it takes a batch of very small size and computes gradient

$$w_{n+1} = w_n - \eta \sum_{j=1}^{M} \nabla_w L_j / M \quad \{M << N\}$$

- convergence rate $O(1/n) = c/n$

→ Epoch: Set of descent steps required to go through all N samples.

- GD : 1 epoch = 1 descent steps
- SGD : 1 epoch = N descent steps
- SGD minibatch : 1 epoch = N/M descent steps

* Stochastic Average Gradient (SAG) :

$$w_{n+1} = w_n - \eta \left(\nabla_w L_j^{(n)} - \nabla_w L_j^{(n-1)} + \sum_{i=1}^{N} \nabla_w L_i^{(n-1)} \right) / N$$

(biased estimate)

* Stochastic Average Gradient Accelerated (SAGA)

$$w_{n+1} = w_n - \eta \left(\nabla_w L_j^{(n)} - \nabla_w L_j^{(n-1)} + \sum_{i=1}^{N} \nabla_w L_i^{(n-1)} / N \right)$$

(unbiased estimate)

- converges at $O(1/n)$

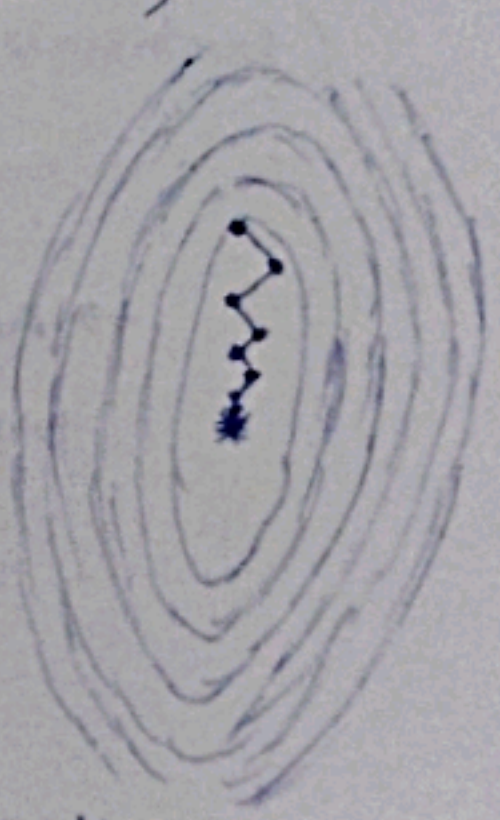
Optimal Learning rate (Steepest Descent) :-

$$\text{Optimal } \eta^* = \arg\min_\eta L(w_{n+1} - \eta \nabla_{w_{n+1}} L)$$

$$= \arg\min_\eta L(w_n - \eta g_{n-1})$$

(Not always computable)

At optimal η^* ; $g_n^T g_{n-1} = 0$



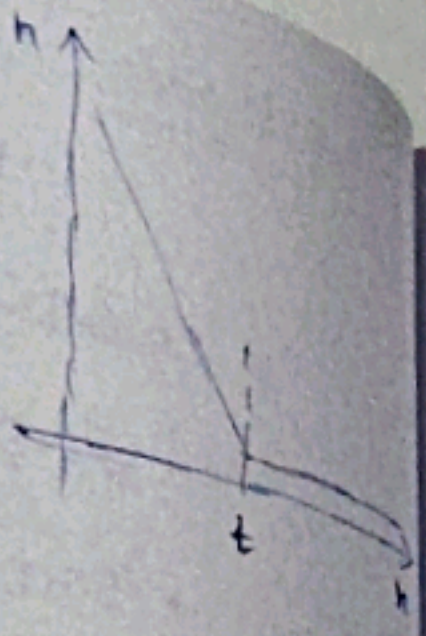
→ consecutive gradients are orthogonal
- Slow convergence, undoing previous minimizations

Adaptive Learning rate

* Linear decay :- $\eta_{n+1} = (1-\alpha)\eta_0 + \alpha\eta_t$

t - terminal rate instance

α - min $(1, n/t)$



* Ada Grad :- $\eta_n = \alpha / (\varepsilon + \sqrt{h_n})$

$$h_n = h_{n-1} + \underbrace{\left(\sum_i \nabla_w L_i^{(n)} / N\right)^2}_{g_n}$$

→ Reduce learning rate, when accumulated gradient is large and vice versa.

→ Good for sparse gradients

→ Decreases learning rate early - vanishing (problem when gradient is noisy)

* RMS Prop :- $\eta_n = \alpha / (\varepsilon + \sqrt{h_n})$

$$h_n = \beta h_{n-1} + (1-\beta)\left(\sum_{i=1}^{N} \nabla_w L_i^{(n)} / N\right)^2$$

→ Accumulation is averaged (weighted average) to prevent early decrease in learning rate.

* Adaptive Moment Estimation (Adam):-

* Adaptive learning rate accelerates gradient descent

$$m_{n+1} = \beta_1 m_{n-1} + (1-\beta_1) g_n$$

* m_n - weighted moving average of the gradients
  - gives smoother trajectory.
  - builds momentum that helps to move out of small wells.

* Momentum can lead to moving past the minima
* When the variance of the gradient is too high, take smaller steps to not miss small-well minima and vice-versa.

* Non-central second moment

$$V_n = \beta_2 V_{n-1} + (1-\beta_2) g_n^2$$

* V_n - weighted moving average of the (uncentered) variance of the gradients.
  - Helps to reduce oscillations

→ Adam Optimizer:-

* Compute m_n and k_n ; $k_n = \frac{m_n}{(1-\beta_1^n)}$

* Compute V_n and S_n ; $S_n = \sqrt{\frac{V_n}{(1-\beta_2^n)}}$

* $w_{n+1} = w_n - \eta\, k_n/(S_n+\varepsilon)$